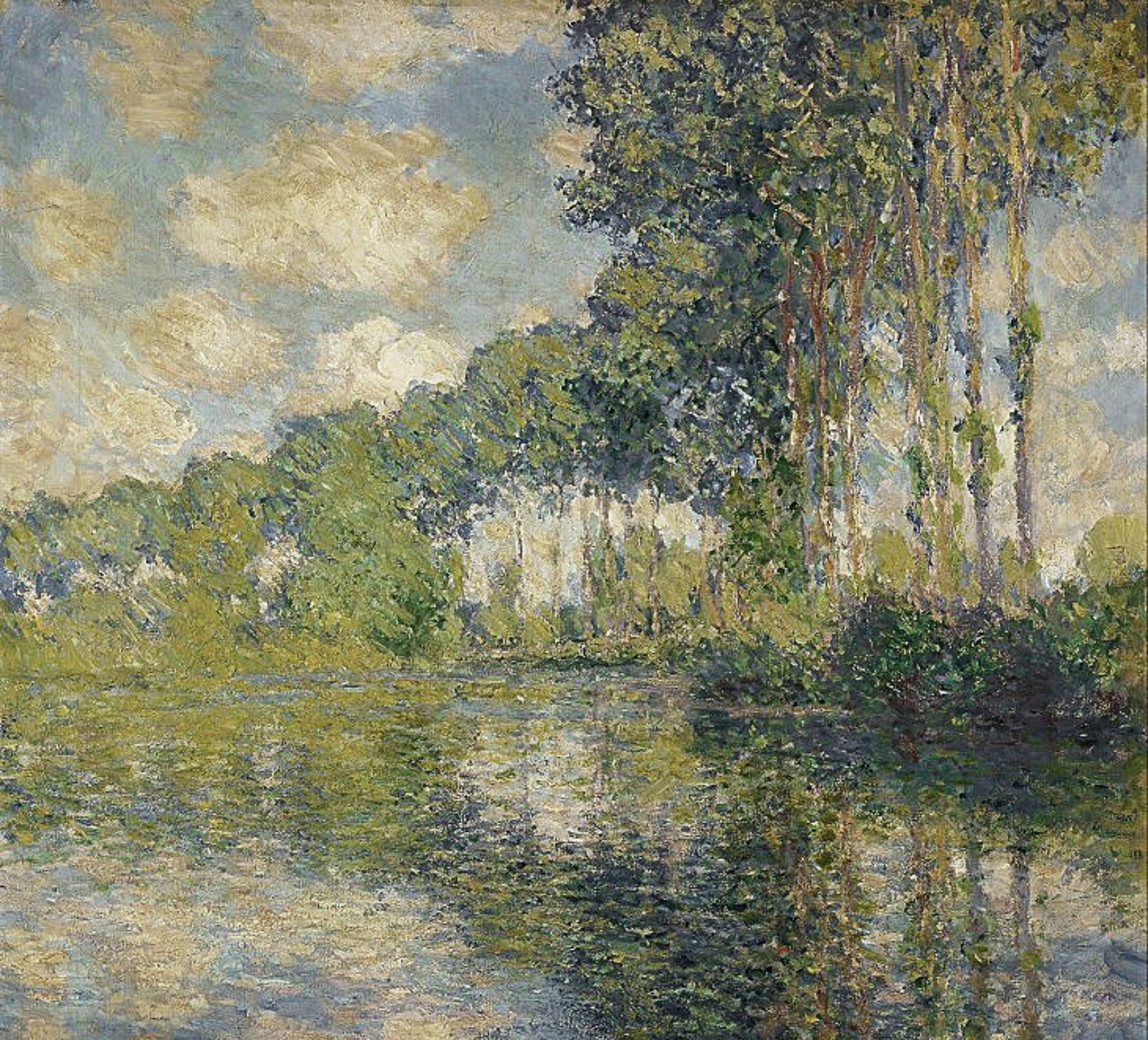




# 智能计算系统实验日志

动手实现深度学习

作者: ZHY

时间: July 21, 2026

版本: 0.1



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡



# 目录

|                                            |          |
|--------------------------------------------|----------|
| <b>第 1 章 基于三层神经网络实现手写数字分类</b>              | <b>1</b> |
| 1.1 实验目标                                   | 1        |
| 1.2 实验环境                                   | 1        |
| 1.3 实验结构与设计                                | 1        |
| 1.3.1 数据加载模块                               | 1        |
| 1.3.2 网络层模块                                | 2        |
| 1.3.2.1 全连接层 (FullyConnectedLayer)         | 2        |
| 1.3.2.2 ReLU 激活层 (ReLULayer)               | 2        |
| 1.3.2.3 Softmax 与交叉熵损失层 (SoftmaxLossLayer) | 2        |
| 1.3.3 模型组装与训练流程                            | 2        |
| 1.4 关键数学原理与推导                              | 2        |
| 1.4.1 全连接层反向传播                             | 2        |
| 1.4.2 ReLU 激活函数                            | 3        |
| 1.4.3 Softmax 与交叉熵的组合梯度                    | 3        |
| 1.4.4 参数更新                                 | 3        |
| 1.5 Python 实现要点与收获                         | 3        |
| 1.5.1 NumPy 矩阵运算与广播                        | 3        |
| 1.5.2 结构化文件解析                              | 3        |
| 1.5.3 面向对象设计与模块导入                          | 4        |
| 1.5.4 其他 Python 技巧                         | 4        |
| 1.6 实验结果与分析                                | 4        |
| 1.7 实验总结                                   | 4        |
| <b>第 2 章 基于 VGG19 实现图像分类</b>               | <b>5</b> |
| 2.1 实验目标                                   | 5        |
| 2.2 实验环境                                   | 5        |
| 2.3 实验结构与设计                                | 5        |
| 2.3.1 网络层模块设计                              | 5        |
| 2.3.1.1 卷积层 (ConvolutionalLayer)           | 5        |
| 2.3.1.2 最大池化层 (MaxPoolingLayer)            | 6        |
| 2.3.1.3 展平层 (FlattenLayer)                 | 6        |
| 2.3.2 VGG19 网络结构                           | 6        |
| 2.3.3 模型参数加载                               | 6        |
| 2.3.3.1 MAT 文件结构解析                         | 6        |
| 2.3.4 图像预处理流程                              | 7        |
| 2.4 关键数学原理与推导                              | 7        |
| 2.4.1 卷积操作的计算复杂度                           | 7        |
| 2.4.2 感受野计算                                | 7        |
| 2.4.3 批量归一化的必要性                            | 7        |
| 2.5 Python 实现要点与收获                         | 7        |
| 2.5.1 卷积层的高效实现                             | 7        |

---

|                        |   |
|------------------------|---|
| 2.5.2 模块化设计 . . . . .  | 8 |
| 2.5.3 文件格式解析 . . . . . | 8 |
| 2.5.4 图像处理 . . . . .   | 8 |
| 2.6 实验结果与分析 . . . . .  | 8 |

# 第 1 章 基于三层神经网络实现手写数字分类

本次实验旨在从零实现一个三层全连接神经网络（多层感知机，MLP），用于解决 MNIST 手写数字的 10 分类问题。实验完全不依赖任何深度学习框架（如 PyTorch、TensorFlow），仅以 NumPy 作为矩阵运算库，完整实现了前向传播、反向传播、参数更新以及数据加载与评估的全流程。

通过本次实验，我们不仅验证了神经网络的基本数学原理，还在工程实现中深入体会了 Python 和 NumPy 的核心特性。

## 1.1 实验目标

1. 理解全连接层、激活函数、损失函数的数学形式及其反向传播的链式法则；
2. 掌握基于批梯度下降的神经网络训练流程；
3. 学会解析 MNIST 的 IDX 二进制文件格式，完成数据预处理；
4. 熟悉 NumPy 的矩阵运算、广播机制、切片操作和随机数生成；
5. 体会面向对象编程在模块化设计中的优势。

## 1.2 实验环境

- 操作系统：Windows 10
- Python 版本：3.13.9
- 核心库：NumPy, struct, os
- 开发环境：Jupyter Notebook

## 1.3 实验结构与设计

整个实验代码分为两个独立的 Jupyter Notebook 文件，体现了良好的模块化思想：

- `layer_1.ipynb`：定义了所有可复用的网络层类（全连接层、ReLU 层、Softmax 损失层），这些类可被其他模型直接导入使用。
- `mnist_mlp_cpu.ipynb`：负责 MNIST 数据的加载、模型的组装、训练循环以及评估，并依赖 `layer_1` 中的层类。

### 1.3.1 数据加载模块

MNIST 数据集以 IDX 二进制格式存储，分为训练图片、训练标签、测试图片、测试标签四个文件。数据加载的核心是 `load_mnist` 方法，它利用 Python 的 `struct` 模块解析文件头（大端序），并读取后续的像素或标签数据。

#### 命题 1.1 (数据归一化)

原始像素值为 0 255 的整数，直接输入神经网络会导致梯度更新不稳定。因此，实验中将所有像素值除以 255，归一化至  $[0, 1]$  区间，以加速收敛。



## 1.3.2 网络层模块

### 1.3.2.1 全连接层 (FullyConnectedLayer)

该层实现了线性变换  $\mathbf{Y} = \mathbf{XW} + \mathbf{b}$ 。其主要方法包括：

- `init_param`: 使用正态分布随机初始化权重，偏置初始化为零。
- `forward`: 执行前向传播，保存输入供反向传播使用。
- `backward`: 根据损失对输出的梯度，计算对权重、偏置和输入的梯度。
- `update_param`: 使用梯度下降更新参数。

### 1.3.2.2 ReLU 激活层 (ReLU Layer)

实现激活函数  $f(x) = \max(0, x)$ ，其导数为  $\mathbf{1}_{x>0}$ 。反向传播时，仅将正输入的梯度传递回去。

### 1.3.2.3 Softmax 与交叉熵损失层 (SoftmaxLossLayer)

该层将最后一层的得分转换为概率分布，并计算交叉熵损失。前向传播采用数值稳定的 Softmax（减去最大值防止溢出）。反向传播时，结合交叉熵损失的梯度简化为：

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y} \quad (1.1)$$

其中  $\mathbf{p}$  为预测概率， $\mathbf{y}$  为 one-hot 编码的真实标签。

## 1.3.3 模型组装与训练流程

在 `mnist_mlp_cpu.ipynb` 中，`MNIST_MLP` 类负责整个训练过程：

1. `load_data`: 加载并归一化数据，将标签拼接到特征矩阵的最后一列。
2. `build_model`: 实例化三层全连接（大小分别为  $784 \rightarrow 32 \rightarrow 16 \rightarrow 10$ ）和两个 ReLU 层，以及最终的 Softmax 损失层。
3. `init_model`: 初始化所有全连接层的参数。
4. `forward / backward / update`: 依次执行前向、反向和参数更新。
5. `train`: 按 epoch 循环，每个 epoch 内打乱数据，按批次训练，并定期输出损失。
6. `evaluate`: 在测试集上计算准确率。

**注**[训练流程设计] 训练采用小批量梯度下降（batch size = 100），每个 epoch 前打乱数据以保证训练样本的随机性。损失函数为交叉熵，优化器为普通 SGD（学习率 0.01）。

## 1.4 关键数学原理与推导

### 1.4.1 全连接层反向传播

设损失函数为  $L$ ，全连接层输入为  $\mathbf{X}$ ，输出为  $\mathbf{Y} = \mathbf{XW} + \mathbf{b}$ 。给定  $\frac{\partial L}{\partial \mathbf{Y}}$ （记作  $\delta$ ），根据链式法则：

**命题 1.2 (全连接层梯度)**

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^\top \delta + \lambda \mathbf{W} \quad (\text{含 L2 正则化}) \quad (1.2)$$

$$\frac{\partial L}{\partial \mathbf{b}} = \sum_{i=1}^B \delta_i \quad (\text{沿批维度求和}) \quad (1.3)$$

$$\frac{\partial L}{\partial \mathbf{X}} = \delta \mathbf{W}^\top \quad (1.4)$$

**注**[梯度维度的对齐] 在代码中, `np.dot(self.input.T, top_diff)` 的结果维度为 (输入维度×输出维度), 正好与权重矩阵一致; `np.sum(top_diff, axis=0, keepdims=True)` 保持偏置为行向量; `np.dot(top_diff, self.weight.T)` 得到与输入同形状的梯度。

## 1.4.2 ReLU 激活函数

### 定理 1.1 (ReLU 的导数)

对于  $f(x) = \max(0, x)$ , 其导数为:

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (1.5)$$

反向传播时, 根据链式法则, 输入梯度为  $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial f} \cdot f'(x)$ , 即仅有正输入部分传递梯度。

## 1.4.3 Softmax 与交叉熵的组合梯度

### 定理 1.2 (Softmax+ 交叉熵的梯度)

设 Softmax 输出为  $\mathbf{p} = \text{softmax}(\mathbf{z})$ , 真实标签 one-hot 为  $\mathbf{y}$ , 交叉熵损失  $L = -\sum_i y_i \log p_i$ , 则:

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y} \quad (1.6)$$

**证明** [推导概要] 由  $\frac{\partial p_i}{\partial z_j} = p_j(\delta_{ij} - p_i)$ , 结合链式法则, 可得上述简洁形式。这使得反向传播实现变得极为简单。

**注**[数值稳定性] 前向传播中, 我们使用  $\text{softmax}(z_i - \max z)$ , 因为 Softmax 对输入平移不变, 且可避免指数溢出。

## 1.4.4 参数更新

采用标准梯度下降:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}, \quad \mathbf{b} \leftarrow \mathbf{b} - \eta \frac{\partial L}{\partial \mathbf{b}} \quad (1.7)$$

其中  $\eta$  为学习率。

# 1.5 Python 实现要点与收获

## 1.5.1 NumPy 矩阵运算与广播

- 批量处理: 所有操作均支持批输入, 输入  $\mathbf{X}$  形状为  $(B, d)$ , 权重  $\mathbf{W}$  为  $(d, h)$ , 利用矩阵乘法一次性计算所有样本的输出。
- 广播机制: 偏置  $\mathbf{b}$  形状为  $(1, h)$ , 与  $(B, h)$  的输出相加时自动广播至每一行, 避免显式复制。
- 高阶索引: 在生成 one-hot 标签时, 使用 `self.label_onehot[np.arange(self.batch_size), label] = 1.0` 实现高效索引赋值。
- 聚合操作: `np.sum(top_diff, axis=0, keepdims=True)` 保持二维结构, 便于后续运算。

## 1.5.2 结构化文件解析

- 使用 `struct.unpack_from` 解析二进制文件头, 指定大端序 ('>') 以正确读取 MNIST 的整数。
- 利用格式化字符串 '>' + `str(data_size)` + 'B' 读取所有像素字节, 并转换为 NumPy 数组。
- 文件读取后使用 `np.reshape` 将一维数据转为二维矩阵 (样本数 × 像素数)。

### 1.5.3 面向对象设计与模块导入

- 将网络层封装为独立的类，每个类实现 `forward` 和 `backward` 接口，便于组合与复用。
- 在 `mnist_mlp_cpu.ipynb` 中通过 `import layer_1 as layer` 导入整个层模块，利用 `importnb` 支持直接导入 Jupyter Notebook 作为模块。
- 参数保存与加载：使用 `np.save` 将参数字典保存为 `.npy` 文件，并通过 `load_model` 恢复。

### 1.5.4 其他 Python 技巧

- 数据打乱： `np.random.permutation` 生成随机索引，并利用花式索引重排数据。
- 切片选择：使用 `train_data[:, :-1]` 获取特征，`train_data[:, -1]` 获取标签。
- 类型转换：将标签强制转为 `np.int64` 以确保索引正确。

## 1.6 实验结果与分析

实验设定：隐藏层大小 32 和 16，学习率 0.01，批大小 100，迭代 20 个 epoch。  
运行输出显示损失从初始约 242 缓慢下降，最终测试集准确率为 96.67%。

```
Epoch: 19, Batch: 300, Loss: 4.3961
fc1 weight gradient norm: 0.435903
fc1 weight norm: 13.164493
fc1 bias gradient norm: 0.041371
Epoch: 19, Batch: 400, Loss: 2.3794
fc1 weight gradient norm: 1.219793
fc1 weight norm: 13.190733
fc1 bias gradient norm: 0.117294
Epoch: 19, Batch: 500, Loss: 12.3316
Accuracy in test set: 0.966700
```

图 1.1: 模型输出

## 1.7 实验总结

本次实验成功搭建了一个完整的神经网络训练框架，涵盖数据加载、模型定义、前向反向传播及参数更新。我们深入理解了全连接层、ReLU、Softmax 的数学原理及其 Python 实现，并熟练运用了 NumPy 的高级操作。通过本次实验：

- 巩固了链式法则在神经网络中的应用；
- 强化了矩阵维度分析的能力；
- 体验了从零实现深度学习模型的全流程。

## 第 2 章 基于 VGG19 实现图像分类

### 2.1 实验目标

1. 理解 VGG19 卷积神经网络的结构原理与设计思想；
2. 掌握卷积层、池化层、全连接层的前向传播实现；
3. 学习从预训练权重文件（.mat）中加载模型参数；
4. 实现完整的图像预处理与分类推理流程；
5. 理解深度学习框架底层实现与手工实现的异同。

### 2.2 实验环境

- 操作系统：Windows 10
- Python 版本：3.13.9
- 核心库：NumPy 1.26.0, SciPy 1.11.0, Pillow 10.0.0
- 预训练模型：VGG19 (ImageNet, MatConvNet 格式)
- 开发环境：Jupyter Notebook

### 2.3 实验结构与与设计

本次实验代码分为三个核心模块，分别对应三个 Jupyter Notebook 文件：

- layer\_1.ipynb: 定义全连接层、ReLU 激活层和 Softmax 损失层；
- layer\_2.ipynb: 定义卷积层、最大池化层和展平层；
- vgg\_cpu.ipynb: 组装 VGG19 模型、加载预训练参数并执行推理。

#### 2.3.1 网络层模块设计

##### 2.3.1.1 卷积层（ConvolutionalLayer）

卷积层实现了二维互相关运算，是卷积神经网络的核心操作。

###### 定义 2.1 (卷积操作)

对于输入张量  $\mathbf{X} \in \mathbb{R}^{N \times C_{in} \times H \times W}$ ，卷积核  $\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in} \times K \times K}$  和偏置  $\mathbf{b} \in \mathbb{R}^{C_{out}}$ ，输出特征图  $\mathbf{Y}$  的第  $n$  个样本、第  $c$  个通道、位置  $(i, j)$  的值为：

$$Y_{n,c,i,j} = b_c + \sum_{c'=1}^{C_{in}} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} X_{n,c',i \cdot S + u, j \cdot S + v} \cdot W_{c,c',u,v} \quad (2.1)$$

其中  $S$  为步长 (stride)。

###### 命题 2.1 (输出尺寸计算)

给定输入尺寸  $H_{in} \times W_{in}$ ，卷积核大小  $K$ ，填充  $P$ ，步长  $S$ ，输出尺寸为：

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W_{in} + 2P - K}{S} \right\rfloor + 1 \quad (2.2)$$

**注**[代码实现] 在实现中，首先对输入进行零填充 (padding)，然后通过四重循环遍历批次、输出通道、高度和宽度，提取对应的感受野 (patch) 并与卷积核进行逐元素相乘后求和。



### 2.3.1.2 最大池化层 (MaxPoolingLayer)

最大池化层通过下采样减少特征图的空间尺寸，增强模型的平移不变性。

#### 定义 2.2 (最大池化)

在池化窗口  $\mathcal{R}$  内取最大值：

$$Y_{n,c,i,j} = \max_{(u,v) \in \mathcal{R}} X_{n,c,i \cdot S + u, j \cdot S + v} \quad (2.3)$$



**注**[索引记录] 前向传播时需记录最大值的位置索引 ( $\arg \max$ )，以便在训练阶段反向传播时梯度能够精准传递到对应的位置。

### 2.3.1.3 展平层 (FlattenLayer)

展平层将卷积输出的多维张量转换为一维向量，作为全连接层的输入。

#### 命题 2.2 (维度变换)

将形状为  $[N, C, H, W]$  的张量变换为  $[N, C \times H \times W]$ ，其中  $N$  为批次大小。



## 2.3.2 VGG19 网络结构

VGG19 由 5 个卷积块和 3 个全连接层组成，共包含 19 个权重层（16 个卷积层 + 3 个全连接层）。

表 2.1: VGG19 网络结构

| 模块      | 层名称                                       | 输入/输出通道                                     | 输出尺寸                                            |
|---------|-------------------------------------------|---------------------------------------------|-------------------------------------------------|
| Block 1 | conv1_1, relu1_1, conv1_2, relu1_2, pool1 | 3 $\rightarrow$ 64                          | 224 $\times$ 224 $\rightarrow$ 112 $\times$ 112 |
| Block 2 | conv2_1, relu2_1, conv2_2, relu2_2, pool2 | 64 $\rightarrow$ 128                        | 112 $\times$ 112 $\rightarrow$ 56 $\times$ 56   |
| Block 3 | conv3_1-4, relu3_1-4, pool3               | 128 $\rightarrow$ 256                       | 56 $\times$ 56 $\rightarrow$ 28 $\times$ 28     |
| Block 4 | conv4_1-4, relu4_1-4, pool4               | 256 $\rightarrow$ 512                       | 28 $\times$ 28 $\rightarrow$ 14 $\times$ 14     |
| Block 5 | conv5_1-4, relu5_1-4, pool5               | 512 $\rightarrow$ 512                       | 14 $\times$ 14 $\rightarrow$ 7 $\times$ 7       |
| FC      | fc6, relu6, fc7, relu7, fc8, softmax      | 25088 $\rightarrow$ 4096 $\rightarrow$ 1000 | -                                               |

## 2.3.3 模型参数加载

### 2.3.3.1 MAT 文件结构解析

预训练权重以 MATLAB 的 .mat 格式存储，包含以下关键字段：

- **layers**: 包含所有网络层的参数
- **normalization**: 包含图像均值

每个卷积层的数据结构为结构化数组，包含以下字段：

```
layer = {
    'weights': [weight_array, bias_array],
    'pad': padding_value,
    'type': 'conv',
    'name': 'conv1_1',
    'stride': stride_value
}
```

**命题 2.3 (权重维度转换)**

VGG19 的 MAT 文件中，卷积核权重存储为  $[H, W, C_{in}, C_{out}]$  形状，而我们的实现期望  $[C_{in}, H, W, C_{out}]$  形状，因此需要进行转置：

$$\mathbf{W}_{\text{ours}} = \text{transpose}(\mathbf{W}_{\text{mat}}, (2, 0, 1, 3)) \quad (2.4)$$



**注**[全连接层权重] 全连接层在 MAT 文件中存储为  $[1, 1, C_{in}, C_{out}]$  的四维张量，需要 reshape 为  $[C_{in}, C_{out}]$  的二维矩阵。特别地，fc6 层的权重形状为  $(7, 7, 512, 4096)$ ，展平后为  $(25088, 4096)$ 。

### 2.3.4 图像预处理流程

1. 读取图像：使用 PIL 库读取图像文件
2. 尺寸调整：将图像缩放至  $224 \times 224$  像素
3. 数据类型转换：转换为 float32 类型
4. 减均值：减去 ImageNet 数据集的均值（每通道）
5. 维度调整： $[H, W, C] \rightarrow [1, C, H, W]$

## 2.4 关键数学原理与推导

### 2.4.1 卷积操作的计算复杂度

**定理 2.1 (卷积计算量)**

对于输入尺寸  $H_{in} \times W_{in}$ ，卷积核大小  $K \times K$ ，输入通道数  $C_{in}$ ，输出通道数  $C_{out}$ ，一次卷积的计算量为：

$$\text{FLOPs} = 2 \times H_{out} \times W_{out} \times C_{out} \times C_{in} \times K^2 \quad (2.5)$$



### 2.4.2 感受野计算

**命题 2.4 (感受野递推)**

对于第  $l$  层的感受野  $RF_l$ ，有递推关系：

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod_{i=1}^{l-1} S_i \quad (2.6)$$

其中  $K_l$  为第  $l$  层的卷积核大小， $S_i$  为第  $i$  层的步长。



**例题 2.1 VGG19 的感受野** VGG19 采用  $3 \times 3$  卷积核和步长 2 的池化，经过 5 个池化层后，最后的感受野达到  $196 \times 196$ ，几乎覆盖整个  $224 \times 224$  输入图像。

### 2.4.3 批量归一化的必要性

**注**[预训练权重的优势] 使用 ImageNet 预训练权重可以显著提升模型在图像分类任务上的性能，因为模型已经学习到了通用的视觉特征（如边缘、纹理、物体部件等），这些特征可以作为良好的初始先验。

## 2.5 Python 实现要点与收获

### 2.5.1 卷积层的高效实现

- 使用填充（padding）保持空间分辨率

- 通过四重循环实现卷积，便于理解但效率较低
- 权重形状需与 MAT 文件格式匹配
- 使用 `np.sum(patch * self.weight[idxc])` 实现逐元素相乘后求和

## 2.5.2 模块化设计

- 每个网络层独立封装为类，实现 `forward` 接口
- VGG19 类负责组装各层并管理前向传播流程
- 参数加载与模型结构分离，便于调试

## 2.5.3 文件格式解析

- 使用 `scipy.io.loadmat` 读取 MATLAB 格式文件
- 通过结构化数组字段名提取权重和偏置
- 处理不同层的参数形状差异

## 2.5.4 图像处理

- 使用 PIL 替代已弃用的 `scipy.misc`
- 正确处理 RGB 图像的通道顺序
- 减去 ImageNet 均值进行中心化

## 2.6 实验结果与分析



图 2.1: 模型输入

```
=====  
Classification Results:  
=====
```

|                                                      |                   |
|------------------------------------------------------|-------------------|
| 1. maze, labyrinth                                   | Confidence: 0.62% |
| 2. patio, terrace                                    | Confidence: 0.56% |
| 3. punching bag, punch bag, punching ball, punchball | Confidence: 0.56% |
| 4. pedestal, plinth, footstall                       | Confidence: 0.51% |
| 5. web site, website, internet site, site            | Confidence: 0.50% |

```
=====
```

图 2.2: 模型输出